

Android Layouts Assignment Report

This updated report includes four Android UI examples prepared using **ConstraintLayout**, **RelativeLayout**, **LinearLayout**, and **FrameLayout**. Each section contains a preview, XML layout code, Kotlin activity code, and a short explanation of how the selected layout manager works.

Student Name:

Index Number:

Module: Mobile Application Development

Submission Date: 08 April 2026

ConstraintLayout

RelativeLayout

LinearLayout

FrameLayout

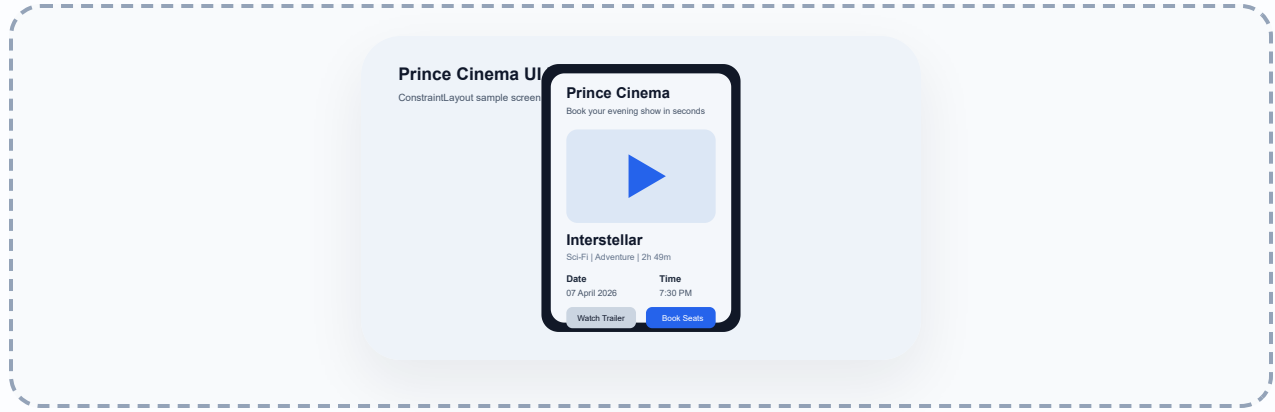
Assignment Overview

The sample application theme used in this report is **Prince Cinema**. The idea was to build a different user interface for each Android layout type while keeping the overall domain consistent. This makes it easier to compare how the same mobile application concept can be implemented with different layout managers.

The four selected examples demonstrate different strengths. ConstraintLayout shows flexible responsive positioning, RelativeLayout focuses on view-to-view relationships, LinearLayout demonstrates ordered rows and columns, and FrameLayout highlights layered content. Together they provide a practical comparison for Android UI development.

1. ConstraintLayout

This sample screen presents a movie booking summary with aligned labels, a poster area, and two action buttons arranged using constraints.



Why this fits: ConstraintLayout is used here to position each view precisely against the parent, a guideline, and neighboring elements without deep nesting.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F4F7FB"
    android:padding="20dp"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/tvAppName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Prince Cinema"
        android:textColor="#0F172A"
        android:textSize="28sp"
        android:textStyle="bold"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

```
<TextView
    android:id="@+id/tvWelcome"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Book your evening show in seconds"
    android:textColor="#475569"
    android:textSize="16sp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvAppName" />

<ImageView
    android:id="@+id/ivPoster"
    android:layout_width="0dp"
    android:layout_height="220dp"
    android:layout_marginTop="20dp"
    android:background="#DCE7F5"
    android:contentDescription="Movie poster"
    android:padding="40dp"
    android:scaleType="centerInside"
    android:src="@android:drawable/ic_media_play"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvWelcome" />

<TextView
    android:id="@+id/tvMovieName"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="Interstellar"
    android:textColor="#0F172A"
    android:textSize="24sp"
    android:textStyle="bold"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/ivPoster" />

<TextView
    android:id="@+id/tvGenre"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:text="Sci-Fi | Adventure | 2h 49m"
    android:textColor="#64748B"
    android:textSize="14sp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvMovieName" />
```

```
<TextView
    android:id="@+id/tvDateLabel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="Date"
    android:textColor="#1E293B"
    android:textStyle="bold"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvGenre" />

<TextView
    android:id="@+id/tvDateValue"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="07 April 2026"
    android:textColor="#475569"
    android:textSize="15sp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvDateLabel" />

<TextView
    android:id="@+id/tvTimeLabel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Time"
    android:textColor="#1E293B"
    android:textStyle="bold"
    app:layout_constraintBaseline_toBaselineOf="@id/tvDateLabel"
    app:layout_constraintStart_toStartOf="@id/guidelineCenter" />

<TextView
    android:id="@+id/tvTimeValue"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="7:30 PM"
    android:textColor="#475569"
    android:textSize="15sp"
    app:layout_constraintStart_toStartOf="@id/guidelineCenter"
    app:layout_constraintTop_toBottomOf="@id/tvTimeLabel" />

<androidx.constraintlayout.widget.Guideline
    android:id="@+id/guidelineCenter"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
```

```
app:layout_constraintGuide_percent="0.55" />

<Button
    android:id="@+id/btnTrailer"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="28dp"
    android:layout_marginEnd="8dp"
    android:backgroundTint="#CBD5E1"
    android:text="Watch Trailer"
    android:textAllCaps="false"
    android:textColor="#0F172A"
    app:layout_constraintEnd_toStartOf="@id/btnBook"
    app:layout_constraintHorizontal_chainStyle="spread"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvDateValue" />

<Button
    android:id="@+id/btnBook"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="8dp"
    android:layout_marginTop="28dp"
    android:backgroundTint="#2563EB"
    android:text="Book Seats"
    android:textAllCaps="false"
    android:textColor="FFFFFF"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toEndOf="@id/btnTrailer"
    app:layout_constraintTop_toBottomOf="@id/tvTimeValue" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

```
package com.example.constraintuiapp

import android.os.Bundle
import android.widget.Button
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
```

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContentView(R.layout.activity_main)

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { view, insets
->
        val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
        view.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
        insets
    }

    findViewById<Button>(R.id.btnBook).setOnClickListener {
        Toast.makeText(this, "Seat booking opened", Toast.LENGTH_SHORT).show()
    }

    findViewById<Button>(R.id.btnTrailer).setOnClickListener {
        Toast.makeText(this, "Trailer preview selected", Toast.LENGTH_SHORT).show()
    }
}
}
```

About ConstraintLayout

ConstraintLayout is a modern Android layout manager that helps developers design flexible and responsive user interfaces with fewer nested views. Instead of placing one layout inside another many times, ConstraintLayout allows each UI component to be positioned by defining relationships, or constraints, with other components or with the parent layout. For example, a button can be attached below a text view, aligned to the start of the screen, or centered between two sides. This approach makes the layout more organized and improves performance because the view hierarchy stays flatter.

One of the main advantages of ConstraintLayout is its ability to create complex screens in a clean and efficient way. A developer can align views vertically and horizontally, set margins, control size using constraints, and even use guidelines, chains, barriers, and bias values to fine-tune the design. This gives more control when compared with simple linear or relative layouts. It is especially useful when building screens that need to look good on many different Android devices with different screen sizes and resolutions.

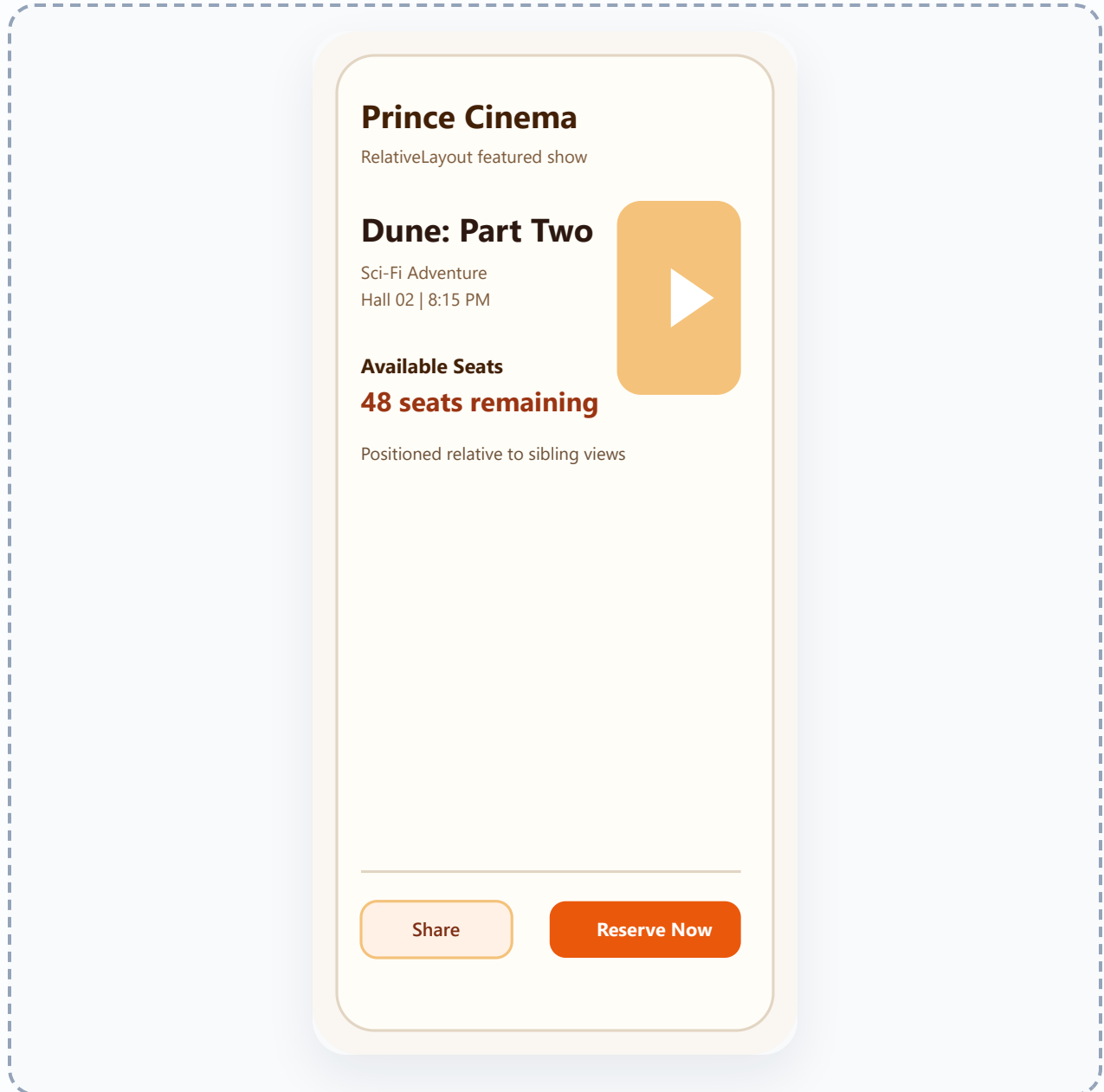
Another important benefit of ConstraintLayout is that it works very well with the Android Studio Design Editor. Developers can drag and drop buttons, text views, images, and other widgets onto the screen, then connect them visually using constraints. This makes

interface design faster and easier to understand, especially for beginners. The generated XML code is also easy to read once the constraint rules are understood.

In summary, ConstraintLayout is a powerful, efficient, and practical layout system for Android application development. It helps create attractive user interfaces, reduces unnecessary nesting, supports responsive design, and offers excellent tool support inside Android Studio. Because of these strengths, it is widely used for modern Android UI design.

2. RelativeLayout

This sample screen shows a featured movie panel where the poster, text block, and action buttons are placed by referring to the parent and to neighboring views.



Why this fits: RelativeLayout makes it easy to say that one view should appear below another, beside another, or attached to an edge of the parent.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FAF7F2"
    android:padding="20dp"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/tvBrand"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Prince Cinema"
        android:textColor="#422006"
        android:textSize="28sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvSubtitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/tvBrand"
        android:layout_marginTop="6dp"
        android:text="RelativeLayout demo for a featured show"
        android:textColor="#7C5A3B"
        android:textSize="15sp" />

    <ImageView
        android:id="@+id/ivPoster"
        android:layout_width="128dp"
        android:layout_height="188dp"
        android:layout_below="@id/tvSubtitle"
        android:layout_alignParentEnd="true"
        android:layout_marginTop="22dp"
        android:background="#F4C27A"
        android:contentDescription="Movie poster"
        android:padding="34dp"
        android:scaleType="centerInside"
        android:src="@android:drawable/ic_media_play" />

    <TextView
        android:id="@+id/tvMovieTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignTop="@id/ivPoster"
        android:layout_toStartOf="@id/ivPoster">
```

```
android:layout_marginEnd="16dp"
android:text="Dune: Part Two"
android:textColor="#2C1810"
android:textSize="24sp"
android:textStyle="bold" />
```

```
<TextView
```

```
    android:id="@+id/tvMovieInfo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvMovieTitle"
    android:layout_alignStart="@id/tvMovieTitle"
    android:layout_toStartOf="@id/ivPoster"
    android:layout_marginTop="8dp"
    android:layout_marginEnd="16dp"
    android:text="Sci-Fi Adventure | Hall 02 | 8:15 PM"
    android:textColor="#7C5A3B"
    android:textSize="15sp" />
```

```
<TextView
```

```
    android:id="@+id/tvSeatLabel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvMovieInfo"
    android:layout_alignStart="@id/tvMovieInfo"
    android:layout_marginTop="24dp"
    android:text="Available Seats"
    android:textColor="#422006"
    android:textStyle="bold" />
```

```
<TextView
```

```
    android:id="@+id/tvSeatValue"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvSeatLabel"
    android:layout_alignStart="@id/tvSeatLabel"
    android:layout_marginTop="6dp"
    android:text="48 seats remaining"
    android:textColor="#9A3412"
    android:textSize="18sp"
    android:textStyle="bold" />
```

```
<TextView
```

```
    android:id="@+id/tvNote"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvSeatValue"
    android:layout_alignStart="@id/tvSeatValue"
    android:layout_toStartOf="@id/ivPoster"
```

```
android:layout_marginTop="10dp"
android:layout_marginEnd="16dp"
android:text="Views are placed relative to the parent and to sibling elements."
android:textColor="#6B4F36"
android:textSize="14sp" />
```

```
<View
```

```
    android:id="@+id/divider"
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:layout_above="@id/btnShare"
    android:layout_marginBottom="18dp"
    android:background="#E2D5C3" />
```

```
<Button
```

```
    android:id="@+id/btnShare"
    android:layout_width="140dp"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true"
    android:text="Share"
    android:textAllCaps="false" />
```

```
<Button
```

```
    android:id="@+id/btnReserve"
    android:layout_width="160dp"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true"
    android:layout_alignParentEnd="true"
    android:backgroundTint="#EA580C"
    android:text="Reserve Now"
    android:textAllCaps="false"
    android:textColor="#FFFFFF" />
```

```
</RelativeLayout>
```

MainActivity.kt

```
package com.example.relativeuiapp

import android.os.Bundle
import android.widget.Button
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
```

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { view, insets
->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            view.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }

        findViewById<Button>(R.id.btnReserve).setOnClickListener {
            Toast.makeText(this, "Reservation panel opened", Toast.LENGTH_SHORT).show()
        }

        findViewById<Button>(R.id.btnShare).setOnClickListener {
            Toast.makeText(this, "Show details shared", Toast.LENGTH_SHORT).show()
        }
    }
}
```

About RelativeLayout

RelativeLayout is an Android layout manager that places views according to their relationship with the parent container or with other views already inside the screen. Instead of stacking everything in one row or one column, a developer can say that one view should appear below another, to the right of another, aligned to the bottom of the screen, or centered inside the parent. This makes RelativeLayout useful when a design has elements that depend on each other's positions.

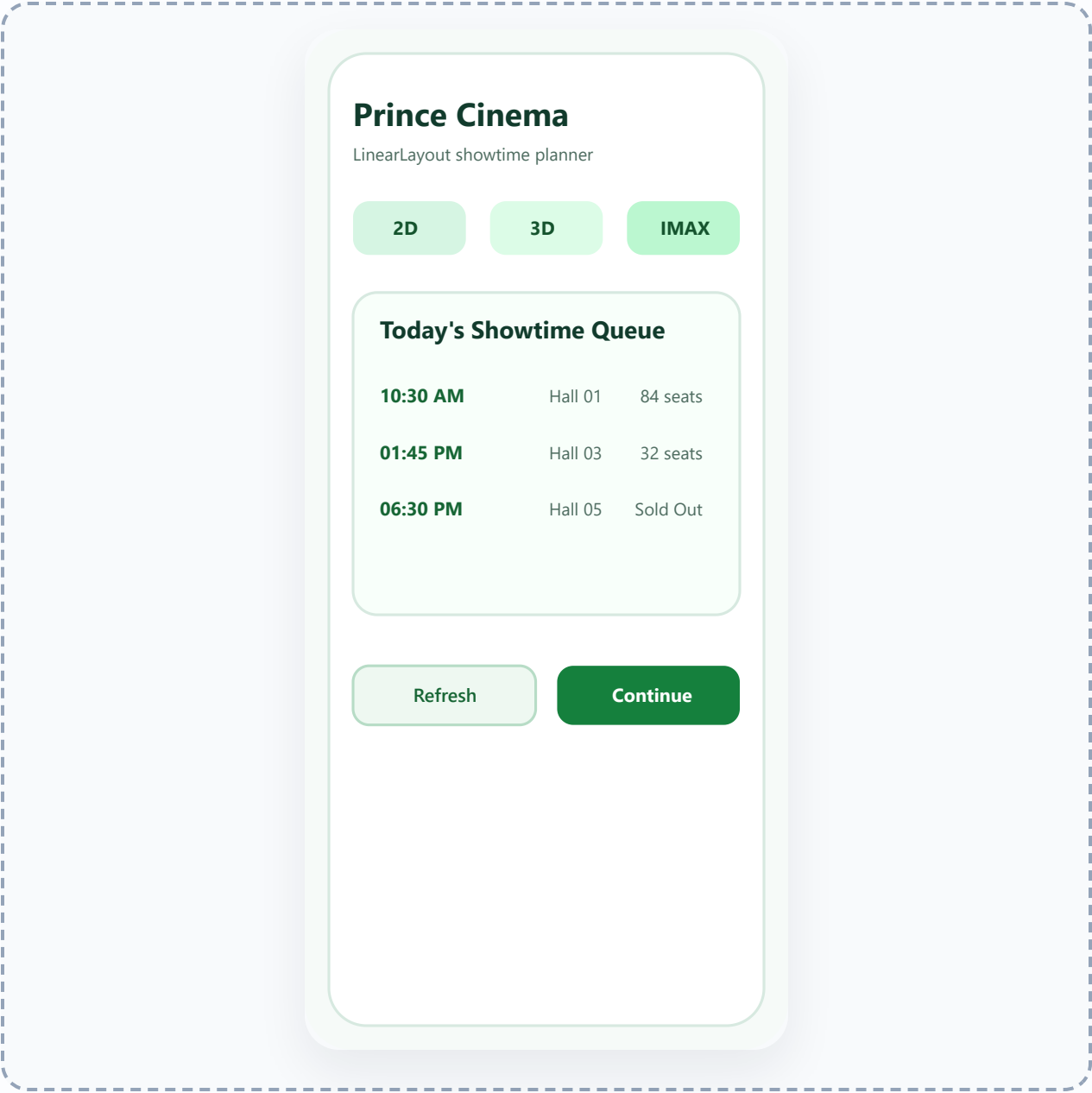
In this sample interface, the movie poster is aligned to the right side of the screen while the title, show information, and seat details are placed to its left. The text section is arranged by using rules such as below and toStartOf, and the action buttons are attached to the bottom of the parent layout. This clearly demonstrates the core idea of RelativeLayout, where each component finds its place by referencing another component or the parent container.

RelativeLayout is easy to understand and was widely used before ConstraintLayout became the preferred choice for modern Android design. It is still helpful for small and medium-sized layouts because the XML remains readable and the positioning rules are

simple. However, for very complex screens, ConstraintLayout is usually more efficient and flexible. Even so, RelativeLayout remains a valuable layout for learning how view relationships work in Android UI design.

3. LinearLayout

This sample screen builds a simple showtime planner using vertical and horizontal rows, making the order of views very easy to follow.



Why this fits: LinearLayout is best when the interface naturally flows in one direction and child views can be grouped into rows or columns.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F6FAF8"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Prince Cinema"
        android:textColor="#113B2C"
        android:textSize="28sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="6dp"
        android:text="LinearLayout demo for showtime planning"
        android:textColor="#4D6B5F"
        android:textSize="15sp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="22dp"
        android:gravity="center"
        android:orientation="horizontal">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginEnd="8dp"
            android:layout_weight="1"
            android:background="#D6F5E3"
            android:gravity="center"
            android:padding="12dp"
            android:text="2D"
            android:textColor="#14532D"
            android:textStyle="bold" />

        <TextView
            android:layout_width="0dp"
```

```
        android:layout_height="wrap_content"
        android:layout_marginHorizontal="4dp"
        android:layout_weight="1"
        android:background="#DCFCE7"
        android:gravity="center"
        android:padding="12dp"
        android:text="3D"
        android:textColor="#14532D"
        android:textStyle="bold" />

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_weight="1"
        android:background="#BBF7D0"
        android:gravity="center"
        android:padding="12dp"
        android:text="IMAX"
        android:textColor="#14532D"
        android:textStyle="bold" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:background="#FFFFFF"
    android:orientation="vertical"
    android:padding="18dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Today's Showtime Queue"
        android:textColor="#113B2C"
        android:textSize="20sp"
        android:textStyle="bold" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="18dp"
        android:gravity="center_vertical"
        android:orientation="horizontal">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
```

```
        android:layout_weight="1"
        android:text="10:30 AM"
        android:textColor="#166534"
        android:textStyle="bold" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hall 01" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="18dp"
    android:text="84 seats" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="14dp"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="01:45 PM"
        android:textColor="#166534"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hall 03" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="18dp"
        android:text="32 seats" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="14dp"
    android:gravity="center_vertical"
```

```
        android:orientation="horizontal">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="06:30 PM"
            android:textColor="#166534"
            android:textStyle="bold" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Hall 05" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="18dp"
            android:text="Sold Out" />
    </LinearLayout>
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:gravity="center"
    android:orientation="horizontal">

    <Button
        android:id="@+id/btnRefresh"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginEnd="8dp"
        android:layout_weight="1"
        android:text="Refresh"
        android:textAllCaps="false" />

    <Button
        android:id="@+id/btnContinue"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_weight="1"
        android:backgroundTint="#15803D"
        android:text="Continue"
        android:textAllCaps="false"
        android:textColor="#FFFFFF" />
```

</LinearLayout>

</LinearLayout>

MainActivity.kt

```
package com.example.linearuiapp

import android.os.Bundle
import android.widget.Button
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { view, insets
->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            view.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }

        findViewById<Button>(R.id.btnRefresh).setOnClickListener {
            Toast.makeText(this, "Showtimes refreshed", Toast.LENGTH_SHORT).show()
        }

        findViewById<Button>(R.id.btnContinue).setOnClickListener {
            Toast.makeText(this, "Moving to seat selection", Toast.LENGTH_SHORT).show()
        }
    }
}
```

About LinearLayout

LinearLayout is one of the simplest Android layout managers because it arranges views in a single direction. By setting the orientation to vertical or horizontal, a developer can

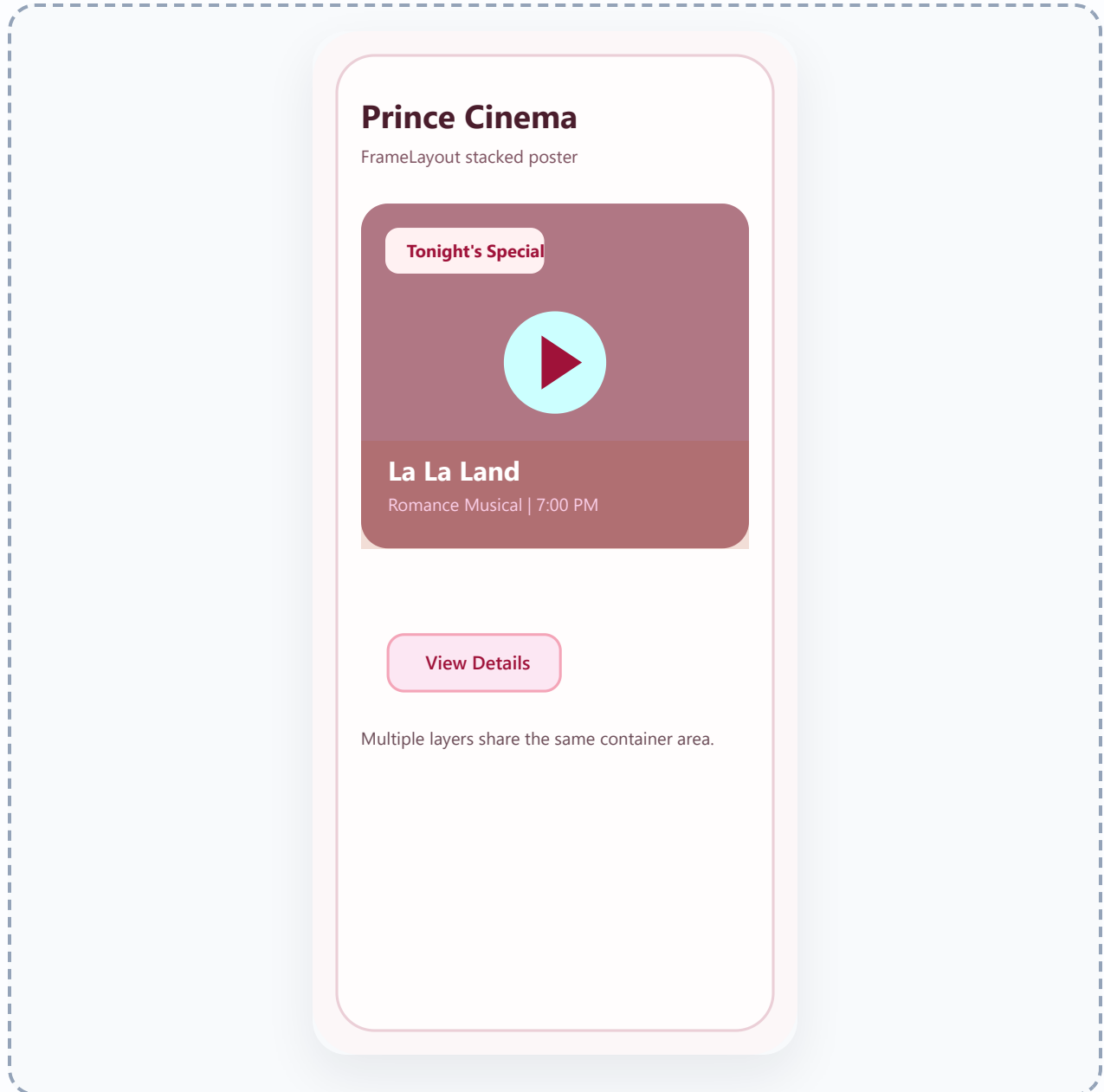
place components in a clear sequence either from top to bottom or from left to right. This makes `LinearLayout` very useful when the interface naturally follows an ordered flow, such as forms, menus, lists of actions, or rows of buttons.

In this sample screen, the content is built using several vertical and horizontal `LinearLayout` containers. The top row displays movie formats, the middle section stacks showtime rows one under another, and the bottom section places two action buttons side by side. Weight values are used to divide horizontal space evenly, which is a common `LinearLayout` technique. The result is a clean structure that clearly shows how rows and columns are created by nesting layouts.

`LinearLayout` is ideal for beginner-friendly Android projects because the XML is easy to read and the behavior is predictable. However, if many `LinearLayout`s are nested inside each other, the view hierarchy can become deep and harder to manage. For simple screens, it remains an excellent choice, but for larger and more flexible designs, developers often move to `ConstraintLayout`. Even then, understanding `LinearLayout` is important because it teaches the basic logic of arranging UI elements in Android.

4. FrameLayout

This sample screen uses one main content area to stack a poster, an overlay, a badge, a play icon, and a bottom information panel.



Why this fits: FrameLayout is designed for layered content, where each new child view can appear on top of the previous one in the same space.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FCF7F8"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Prince Cinema"
        android:textColor="#4A1D2C"
        android:textSize="28sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="6dp"
        android:text="FrameLayout demo for stacked poster content"
        android:textColor="#7D5260"
        android:textSize="15sp" />

    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="260dp"
        android:layout_marginTop="24dp">

        <ImageView
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:background="#F4A5B8"
            android:contentDescription="Poster background"
            android:scaleType="centerCrop"
            android:src="@android:drawable/ic_menu_gallery" />

        <View
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:background="#66000000" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
        android:layout_gravity="top|start"
        android:layout_margin="16dp"
        android:background="#FFF1F2"
        android:paddingLeft="12dp"
        android:paddingTop="8dp"
        android:paddingRight="12dp"
        android:paddingBottom="8dp"
        android:text="Tonight's Special"
        android:textColor="#9F1239"
        android:textStyle="bold" />

<ImageView
    android:id="@+id/btnPlay"
    android:layout_width="72dp"
    android:layout_height="72dp"
    android:layout_gravity="center"
    android:background="#CCFFFFFF"
    android:contentDescription="Play trailer"
    android:padding="18dp"
    android:src="@android:drawable/ic_media_play" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    android:background="#CC4A1D2C"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="La La Land"
        android:textColor="#FFFFFF"
        android:textSize="22sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:text="Romance Musical | 7:00 PM"
        android:textColor="#FBCFE8"
        android:textSize="14sp" />

    <Button
        android:id="@+id/btnDetails"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```

        android:layout_marginTop="12dp"
        android:text="View Details"
        android:textAllCaps="false" />
    </LinearLayout>
</FrameLayout>

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:text="This screen demonstrates layered content by stacking a poster, a
dark overlay, a play icon, a top badge, and a bottom details panel in the same area."
    android:textColor="#6B4C57"
    android:textSize="14sp" />

</LinearLayout>

```

MainActivity.kt

```

package com.example.frameuiapp

import android.os.Bundle
import android.view.View
import android.widget.Button
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { view, insets
->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            view.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }

        findViewById<View>(R.id.btnPlay).setOnClickListener {
            Toast.makeText(this, "Trailer is playing", Toast.LENGTH_SHORT).show()

```

```
    }  
  
    findViewById<Button>(R.id.btnDetails).setOnClickListener {  
        Toast.makeText(this, "Opening movie details", Toast.LENGTH_SHORT).show()  
    }  
}  
}
```

About FrameLayout

FrameLayout is an Android layout manager designed for stacking views on top of each other inside a single area. Every child view shares the same parent space, and the order in which the views are declared determines which one appears in front. Because of this behavior, FrameLayout is perfect for overlays, floating labels, media previews, fragments, loading screens, and simple layered designs.

In this sample UI, the poster background is placed first, then a semi-transparent dark overlay is added, followed by a top badge, a centered play icon, and a details panel at the bottom. All of these components exist inside the same FrameLayout container, so the screen demonstrates the main feature of FrameLayout very clearly. Instead of arranging views side by side, the layout builds depth by placing one view above another.

FrameLayout is simple, lightweight, and very useful when the design focuses on a single content area with multiple layers. It is not the best option for large screens that need complex positioning in many directions, but it works extremely well for stacked content. Learning FrameLayout helps developers understand the visual order of views and how Android can create rich interfaces by combining layers in one container.